Module 1

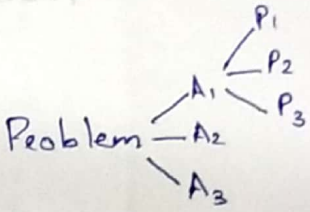
Algorithm - Design

Problem → Solution

Problem → Design Algorithm

Specify the steps to Solve the problem

Step by step procedure



Program → Implementation of an Algorithm in a particular Programming language

Analysis → For selecting the best Algorithm

Characteristics of Algorithm / Criteria

1. Input - Externally supplied (should have 0 or more O/P)
2. Output - Produces atleast one (should have 1 or more O/P)
3. Finiteness - terminate after a particular time (short)
4. Definiteness - Should be very clear and unambiguous
5. Effectiveness - should be feasible

Algorithm → Finite set of instructions that if followed accomplishes a particular task

Criteria: An algorithm can be defined as finite, definite and effective when one or more input are given and atleast one output is produced.

Scanned with CamScanner

Analysis

Problem — Algorithm 1, Algorithm 2, ... Algorithm n

Select the best Algorithm

Method

1. Independent of the machine and its configuration
2. Correlated with the number of inputs
3. Comparison must be very clear

2 Methods used for comparing the different algorithm are time complexity and space complexity

1. Time Complexity

   How many time taken to complete

2. Space Complexity

   Memory space taken to complete a task by the algorithm

It is represented by $S(P) = C + Sp(I)$

- Problem
- C: fixed part — Number of constants, size of the input
- Sp(I): Variable part — Depend on characteristics

Scanned with CamScanner

Example

```
Algorithm abc (a, b, c)
{
return (a*b*c)/(a+b) * 5      } fixed part
}
```

$S(P) = 4 + 0$ (5-1)

$= 4$

II

```
Algorithm abc (a, b, c)
{
S = a+b+c      } variable part
return S
}
```

III

```
Algorithm Sum (a, n)
{
S = 0;
for i = 1 to n
  {
  S = S + a[i];
  }
return S
}
```

Scanned with CamScanner

Time Complexity

∴ Time required to complete the task

T(n) = Compile Time + Run Time

(no: of inputs)

It uses step count method or frequency count method

Frequency Count Method

Counting the no: of steps required

Comments

- Assignments → eg: Sum = a+b
- Comparison
- Iteration

```
Algorithm abc (a, b, c)
{
  return (a+b+c)/(b*c) - 1
}

T(n) = 1
```

```
Algorithm Sum (a, n)
{
  S = 0;        — 1
  for i = 1 to n  → n+1
  {
    S = S + a[i]  — 1
  }
```

Scanned with CamScanner

return s
}

(n+1) + 3

T(n) = n + 4

Divide and Conquer

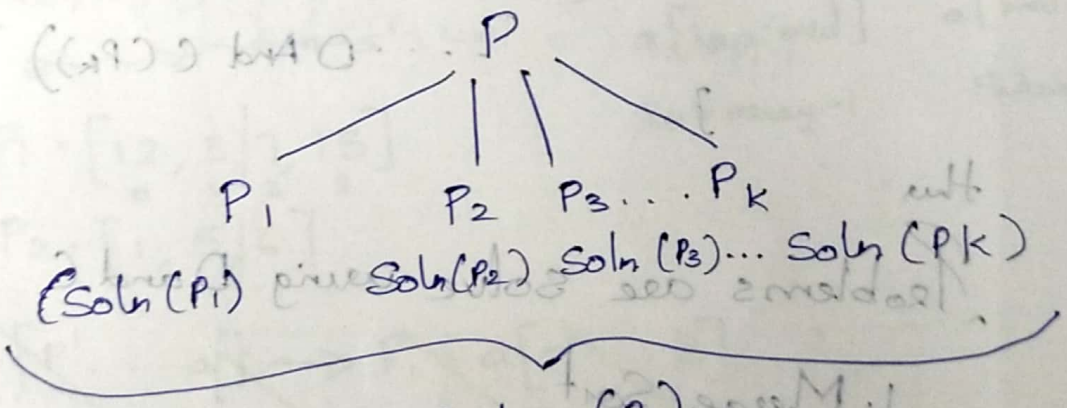
Divide the large problem into different subproblem:

↓

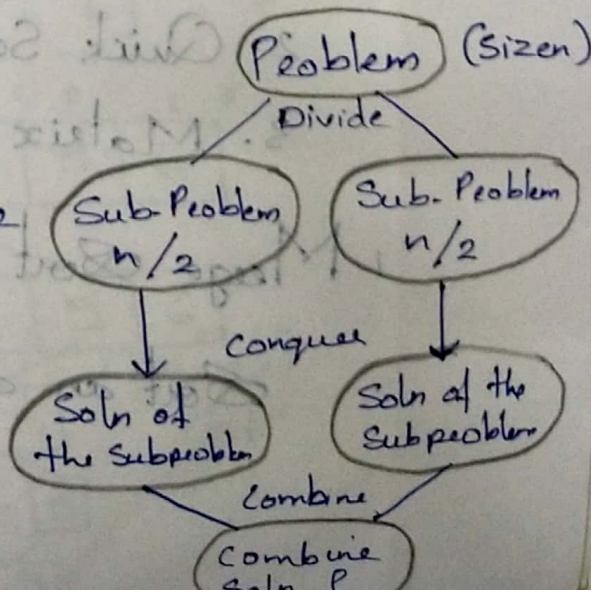
are of the same type as the original problem

Solve the subproblems seperatly → find the subproblems

Combine the soln of the soln of the subproblem to find the soln of the original problem



3 Step 3 → Divide
Conquer
Combine



Scanned with CamScanner

Control Abstraction

Procedure

1. Control flow is clear
2. defined by some procedures whose precise meanings are left undefined.

```
D And C (P)                    Boolean valued function to check whether the problem P is small enough to be solved without splitting
{
  If SMALL (P) then return S(P)        S(P) → Solution to the problem P
  else
  {
    Divide P into k instances P1, P2, ..., Pk,
                                  k ≥ 1
  }
  Return (Combine (D And C (P1), D And C (P2),
                   ... D And C (Pk))
}
```

this

Problems are solve using D and C

1. Merge Sort
2. Quick Sort
3. Matrix Multiplication

1. Merge Sort

Sort an array of n numbers → Problem P

a[0 ... n-1]

a[p ... r]

Scanned with CamScanner

Subproblems

Subarrays → Divide → Find an index q in the midway between p and r (p_1 … p_2)

Sort the subarrays → Conquer → a[p…q] and a[q+1…r]

Merge → Combine → Merge the sorted subarrays to find the final sorted array a[p…r]

The sorted subarray

a[12, 3, 7, 15, 1, 5, 6]
0 1 2 3 4 5 6

a[0…6] n = 7

p = 0

r = 6

q = (p + r)/2

= (0 + 6)/2 = 3

a[p…q] → P_1 = a[0…3]

a[q+1, …r] → P_2 = a[4…6]

a[low:high] - Array

a[low:mid] Sub array-1

a[mid+1:high] Sub array-2

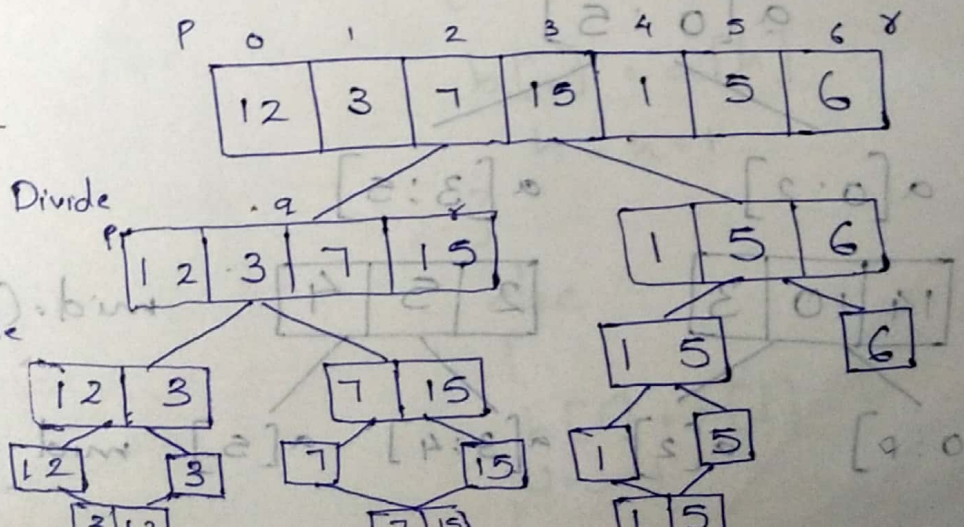
a[12, 3, 7, 15, 1, 5, 6, 2, 4]
0 1 2 3 4 5

P_1 = [12, 3, 7, 15]
0 1 2 3

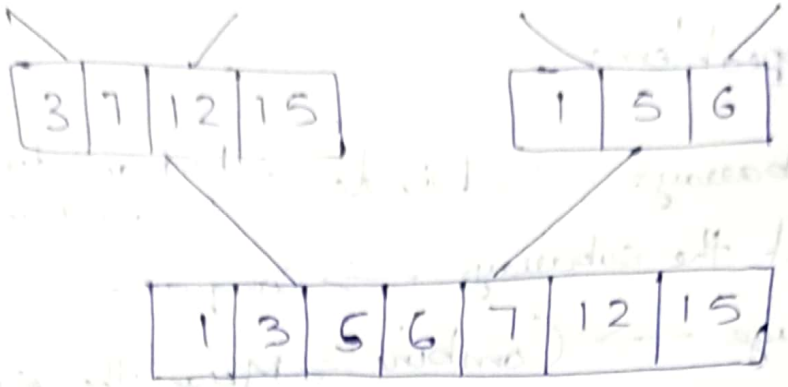
P_2 = [1, 5, 6]

a[p…q] → P_1 = a[0…3]

a[q+1, …r] → P_2 = a[4…6]



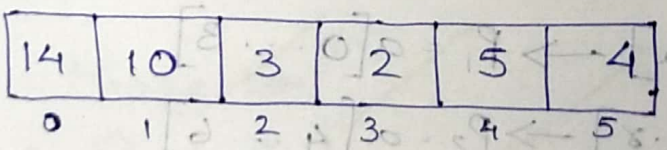
Scanned with CamScanner



: → between element

Algorithm:

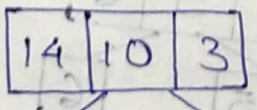
```
MERGESORT (a, low, high) // a [low : high]
{
 if (low < high) // Contains more than one element
                 SMALL (P)
  {
    mid = (low + high) / 2
    MERGESORT (a, low, mid)
    MERGESORT (a, mid + 1, high)
    MERGE (a, low, mid, high)
  }
}
```

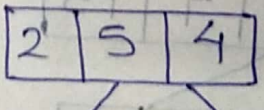


mid = (0+5)/2 = 2

a [0 : 5]

a [0 : 2] a [3 : 5]



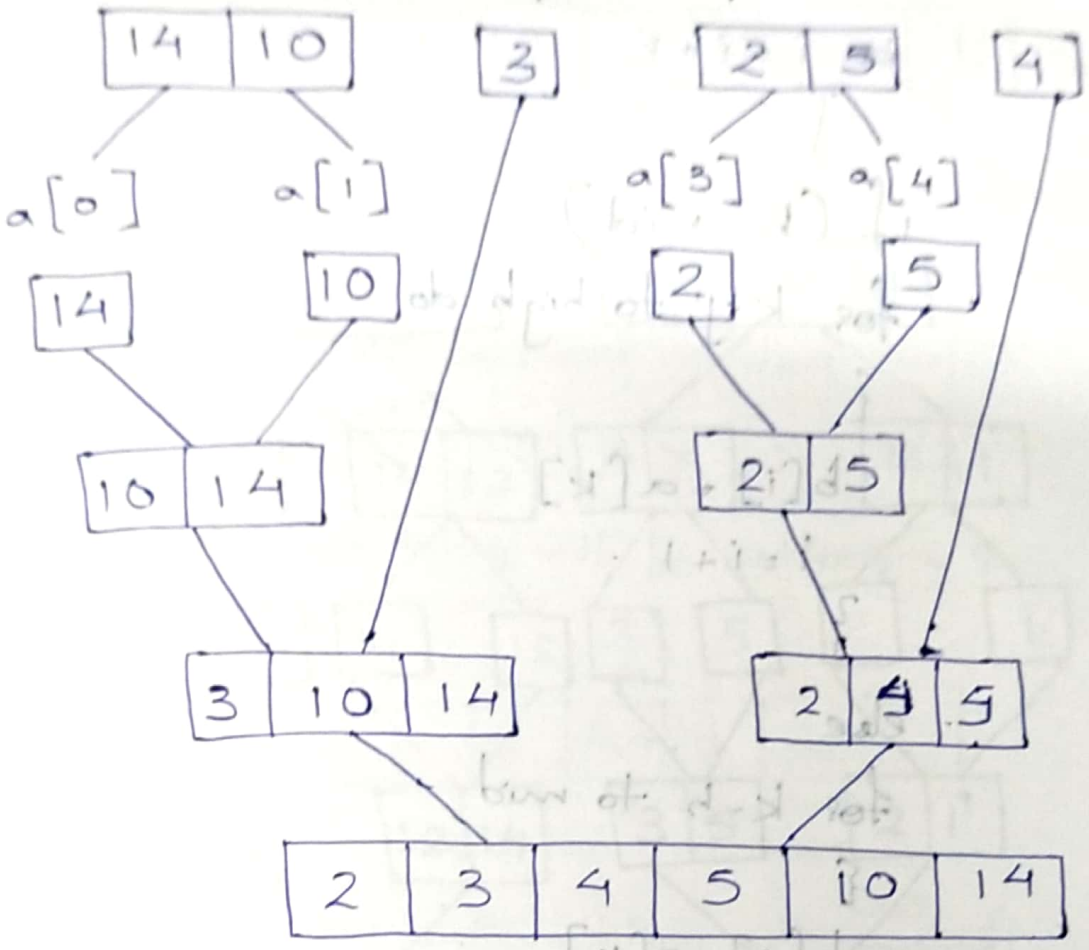


mid = (0+2)/2 = 1

a[0 : 1] a[2] a[3 : 4] a[5]

mid = (3+5)/2 = 8/2 = 4

Scanned with CamScanner

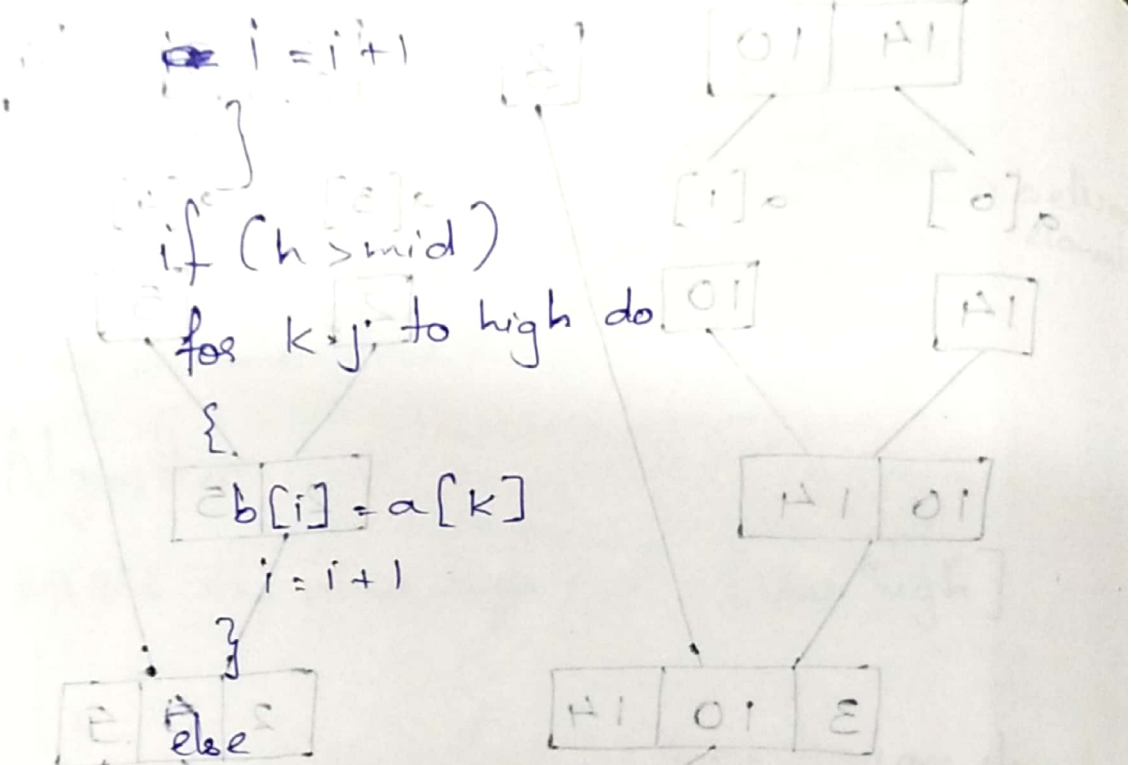


Algorithm MERGE (a, low, mid, high)

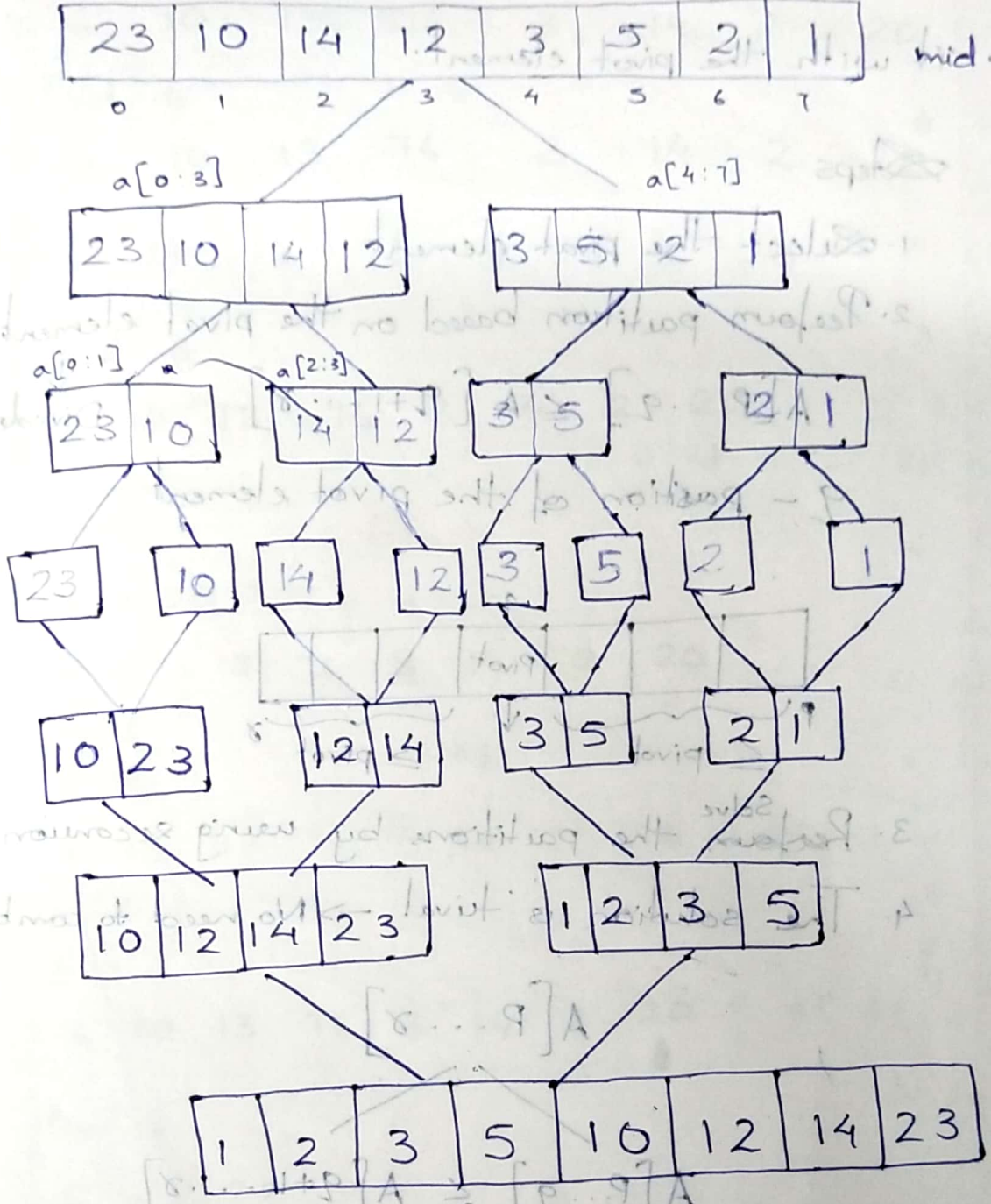
```
{
  h = low, i = low, j = mid + 1
  while (h ≤ mid && j ≤ high)
  {
    if a[h] < a[j]
    {
      b[i] = a[h]
      h = h + 1
    }
    else
    {
      b[i] = a[j]
      j = j + 1
    }
```

Scanned with CamScanner

```
        i = i+1
    }
    if (h > mid)
    for k=j; to high do.
    {
    b[i] = a[k]
      i = i+1
    }
    else
    for k=h to mid
    {
      b[i] = a[k]
      i = i+1
    }
    for i = low to high
    {
    a[i] = b[i]
    }
}
```



Scanned with CamScanner



Quick Sort

- designed using Divide and Conquer (D And C)
- User recursion
- Divide - perform division based on a pivot element
- Conquer - Solving the sub problem
- Combine - There is no need of combining because the solution is trivial

Partition - Exchange Sort

Partition - perform division based on the pivot

Exchange - perform swapping the elements by comparing

Scanned with CamScanner

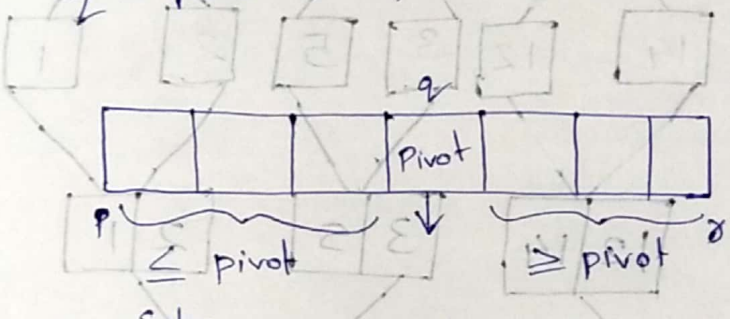
it with the pivot element.

Steps

1. Select the pivot element
2. Perform partition based on the pivot element ⇓ Divide

$A[p \ldots q] \leq A[q+1 \ldots r]$

q - position of the pivot element



3. Solve the partitions by using recursion → Conquer
4. The solution is trival → No need to combine

$A[p \ldots r]$

$A[p \ldots q] \leq A[q+1 \ldots r]$

Partition - 1 $A[q]$ Partition - 2

⇓ Pivot

$A[p \ldots q] \leq A[q+1 \ldots r]$

$A[q] \leq$ pivot

$A[p \ldots q-1] \leq$ Pivot $\leq A[q+1 \ldots r]$

Partition - 1 ⇓ $A[q]$ Partition - 2

Scanned with CamScanner

tep 1 : 6 10 13 76 3 14 2 20

tep 2 : Pivot = 6

6 10 13 76 3 14 2 20

tep 3: 6 10 13 76 3 14 2 20
i

Pivot = 6

tep 4: 6 10 13 76 3 14 2 20
i j

Pivot = 6

tep 5: 6 > 20 ??

6 10 13 76 3 14 2 20
i j

Pivot = 6

tep 6: False

6 10 13 76 3 14 2 20
i j

Pivot = 6

tep 7: j--

6 10 13 76 3 14 2 20
i j

Pivot = 6

Step 8: 6 > 2 ??

6 10 13 76 3 14 2 20
i j

Pivot = 6

Step 9: Tene

6 10 13 76 3 14 2 20
i j

Pivot = 6

Step 10: 6 10 13 76 3 14 2 20
i j

Pivot = 6

Step 11: Pivot = j

Scanned with CamScanner

2 10 13 76 3 14 6 20
i j

Pivot = 6

12) i++

2 10 13 76 3 14 6 20
i j

Pivot = 6

13) 10 > 6 ??

2 10 13 76 3 14 6 20
i j

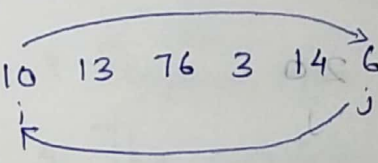
Pivot = 6

14) True

2 10 13 76 3 14 6 20
i j

Pivot = 6

15)



2 10 13 76 3 14 6 20
i j

Pivot = 6

Pivot = i

16) 2 6 13 76 3 14 10 20
i j

Pivot = 6

17) j--

2 6 13 76 3 14 10 20
i j

Pivot = 6

18) 6 > 14 ??

2 6 13 76 3 14 10 20
i j

Pivot = 6

19) False

2 6 13 76 3 14 10 20
i j

Pivot = 6

20) j--

2 6 13 76 3 14 10 20
i j

Pivot = 6

21) 6 > 3 ??

2 6 13 76 3 14 10 20
i j

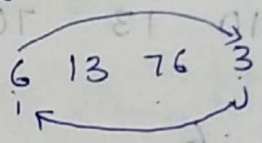
Pivot = 6

22) True

2 6 13 76 3 14 10 20
i j

Pivot = 6

23)



2 6 13 76 3 14 10 20
i j

Pivot = 6

24) Pivot = j

2 3 13 76 6 14 10 20
i j

pivot = 6

25) i++

2 3 13 76 6 14 10 20
i j

Pivot = 6

26)

Quick Sort - Algorithm

```
Algorithm QuickSort (A, P, r)
{
  if (P<r)
    q = Partition (A, P, r)
    Quicksort (A, P, q-1)
    Quicksort (A, q+1, r)
}

Algorithm Partition (A, P, r)
{
  pivot = A[P]
  i = P-1
  j = r-1
  while ~~(P<r)~~ TRUE
  {
   do repeat
      i = i+1 until A[i] ≥ pivot

   do repeat
      j = j-1 until A[j] ≤ pivot

   if (i<j)
  {
  Swap( A[i] & A[j])
  }else
  Swap(A[j] & A[P])
  {return j
  }
```

Scanned with CamScanner

Greedy Method

- Straight forward algorithm design technique
- Problem → n inputs → Candidate set
  - Feasible Solution → Subset that satisfies the problem constraint

- Optimal Solution

Feasible solution that either maximizes / minimizes the objective function.

- Problem will be solved step by step by taking one input at a time
- Select → If it leads to the optimal solution
  Reject → otherwise

Five Component

1. Candidate Set → Set from which the input will be selected
2. Selection Function → Used to select the best candidate
3. Feasible Solution → Solution that satisfies the constraint
4. Objective Function → The function which is

Scanned with CamScanner

used to find the optimal solution

5. Solution Function → The solution that produces the optimal solution

Control Abstraction

```
- Algorithm Greedy (a, n)
  {
   Solution = ϕ ——→ for (i = 1 to n) do
   x = select (a, n) {
   If (Feasible (Solution, x)) then
     {
      Solution = union (Solution, x)
     }
   else
     reject
   }
   return Solution
  }
```

Problems

1. Knapsack
2. Minimum Spanning Tree
   - Prim's
   - Kruskal's
3. Job Sequencing with dead line

Scanned with CamScanner

Knapsack

P - Profit
w - weight
x_i - Part of ith item
n - no of item
m - maximum capacity

$$\max \sum_{0 \le i < n} P_i x_i$$

S.t

$$\sum_{0 \le i < n} w_i x_i \le m$$

$$0 \le x_i <= 1$$

$$P_i \ge 0,\ w_i \ge 0,\ 0 \le i < n$$

Example

$n = 3,\ m = 20\ \ P_1, P_2, P_3 = 25, 24, 15$

$w_1, w_2, w_3 = 18, 15, 10$

Item	1	2	3
P_i	25	24	15
w_i	18	15	10
P_i/w_i	1.36	1.6	1.5

Reassange it

Item	2	3	1
P_i	24	15	25
w_i	15	10	18
P_i/w_i	1.6	1.5	1.38

$x_2 = 1$ (if we can select full x is assigned value

$P_i x_i = 24 \times 1 = 24$

$w_i x_1 = 15 \times 1 = 15 \le 20$

$x_3 = 1/2$ (if we can select half x is assigned value 0

Scanned with CamScanner

$P_i x_i = 15 \times \frac{1}{2} = 7.5$

$w_i x_i = 10 \times \frac{1}{2} = 5$

$\{x_1, x_2, x_3\}$ $\{0, 1, 1/2\}$

Total profit = $0 + 24 + 7.5$

$= 31.5$

2) $P_1, P_2, P_3 = 500, 900, 700$

$w_1, w_2, w_3 = 1, 3, 7$

$m = 5$

$n = 3$

Item	1	2	3
P_i	500	900	700
w_i	1	3	7
P_i/w_i	500	300	100

Item

$x_1 = 1$

$P_i x_i = 500 \times 1 = 500$

$w_i x_i = 1 \times 1 = 1 \leq 5$

$x_2 = 1$

$P_i x_i = 900 \times 1 = 900$

$w_i x_i = 3 \times 1 = 3 \leq 5$

$x_3 = 1/7$

$P_i x_i = 700 \times \frac{1}{7} = 100$

$w_i x_i = 7 \times \frac{1}{7} = 1$

$\{x_1, x_2, x_3\}$ $\{1, 1, 1/7\}$

Total Profit = $500 + 900 + 100 = 500$

Scanned with CamScanner

Algorithm

```
Algorithm Greedy Knapsack (m, n)
// P[1:n] and w[1:n] contain the profit and
weights respectively of items ordered so that
P[i]/w[i] > P[i+1]/w[i+1]
// m is the knapsack size and x[1:n] is the
solution vector
{
for i:=1 to n do x[i]:=0.0 // initialize x
U:=m;
for i:=1 to n do
{
if (w[i] > U) then break;
x[i]:=1.0;
U:=U-w[i];
}
if (i<=n) then x[i]:=U/w[i];
}
```

3 $P_1, P_2, P_3, P_4, P_5 = 6, 14, 6, 9, 5$

$m = 15$

Item	1	2	3	4	5
P_i	6	14	6	9	5
w_i	5	7	2	3	4
P_i/w_i	1.2	2	3	3	1.2

Scanned with CamScanner

Item	3	4	2	1	5
P_i	6	9	14	6	5
w_i	2	3	7	5	4
P_i/w_i	3	3	2	1.2	1.2

$X_3 = 1$

$P_i x_i = 6 \times 1 = 6$

$w_i x_i = 2 \times 1 = 2$

$X_4 = 1$

$P_i x_i = 9 \times 1 = 9$

$w_i x_i = 3 \times 1 = 3$

$X_2 = 1$

$P_i x_i = 14 \times 1 = 14$

$w_i x_i = 7 \times 1 = 7$

$X_1 = 1$

$P_i x_i = 6 \times 1 = 6$

$w_i x_i = 5 \times 1 = 5$

$X_5 = 3/4$

$P_i x_i = 5 \times \frac{3}{4} = \frac{15}{4} = 3.7$

$w_i x_i = 4 \times \frac{3}{4} = 3$

$\{x_1, x_2, x_3, x_4, x_5\} = \{1, 1, 1, 1, 3/4\}$

Total Profit $= 6 + 14 + 6 + 9 + 3.7 = \underline{\underline{38.7}}$

Scanned with CamScanner

4

Item	A	B	C	D
P_i	280	100	120	120
w_i	40	10	20	24
P_i/w_i	7	10	6	5

$m = 60$

Item	B	A	C	D
P_i	100	280	120	120
w_i	10	40	20	24
P_i/w_i	10	7	6	5

$X_B = 1$

$P_i x_i = 100 \times 1 = 100$

$w_i x_i = 10 \times 1 = 10$

$X_A = 1$

$P_i x_i = 280 \times 1 = 280$

$w_i x_i = 40 \times 1 = 40$

$X_C = 1/2$

$P_i x_i = 120 \times 1/2 = 60$

$w_i x_i = 20 \times 1/2 = 10$

$X_D = 0$

$\{x_A, x_B, x_C, x_D\} = \{1, 1, 1/2, 0\}$

Total profit $= 100 + 280 + 60 = 500$

Scanned with CamScanner

Job Sequencing With Deadline

Problem statement

In job sequencing problem, the objective is to find a sequence of jobs, which is completed with their deadlines and gives maximum profit.

Solution

Let us consider, a set of n given jobs which are associated with deadlines $(d_1, d_2, \ldots d_n)$ and profit $(P_1, P_2, \ldots P_n)$ is earned, if a job is completed by its deadline.

These jobs need to be ordered in such a way that there is maximum profit.

It is also given that every job takes single unit of time, so the minimum possible deadline for any job is 1.

How to maximize total profit if only one job can be scheduled at a time.

Here,

We are given a set of jobs

Each job has a defined deadline and some profit associated with it.

The profit of a job is given only when that job is

comp

Greedy Algorithm

It is adopted to determine how the next job is selected for an optimal solution.

The greedy algorithm

Sort

Step-01

Sort all the given jobs in decreasing order of their profits.

Jobs	J_4	J_1	J_3	J_2	J_5	J_6
Deadlines	2	5	3	3	4	2
Profits	300	200	190	180	120	100

Scanned with CamScanner

Step 2

Value of maximum deadline = 5

So, draw a gantt chart with maximum time on Gantt chart - 5 units are shown -

Scanned with CamScanner

Minimum Spanning Tree

Algorithms

1. Kruskal's Algorithm
2. Prim's Algorithm

→ Given a connected and undirected graph, G(V, E), a spanning tree of that graph is a subgraph that is a tree and connects all the vertices together.

A single graph can have many different spanning trees.

A minimum spanning tree (MST) or minimum weight spanning tree for a weighted, connected and undirected graph is a spanning tree with weight less than or equal to the weigh

Kruskal's algorithm

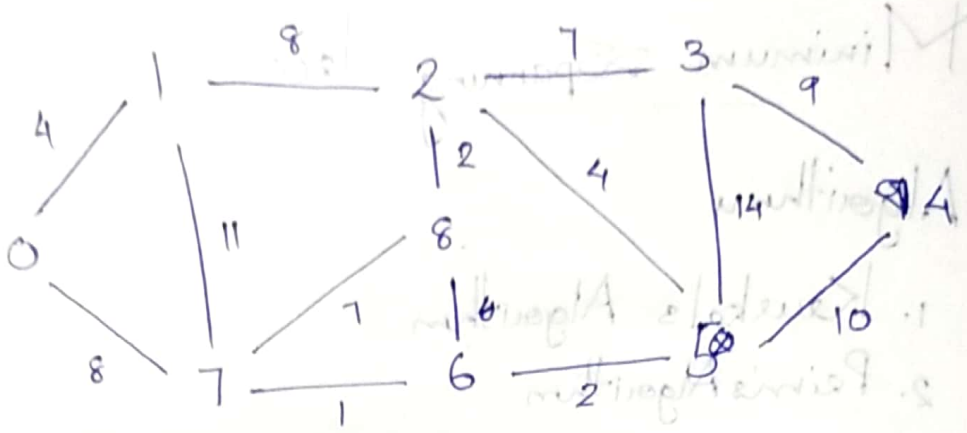
Steps for finding MST

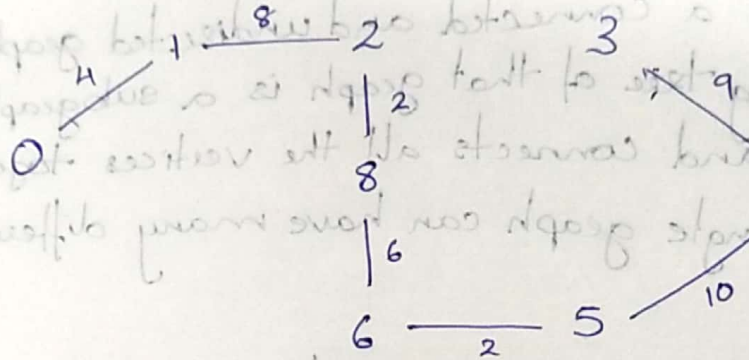
1. Sort all the edges in non-decreasing (increasing) order of their weight.
2. Pick the smallest edge. Check if it forms a cycle with the spanning tree formed so far. If cycle is not formed, include this edge. Else, discard it.
3. Repeat step #2 until there are (V-1) edges in the spanning tree

Example : Algorithm is a Greedy Algorithm

The greedy choice is to pick the smallest weight edge that does not form a cycle in the MST constructed so far.

Scanned with CamScanner





The graph contains 9 vertices and 14 edges. So, the minimum spanning tree formed will be having (9-1) = 8 edges

Weight	Source	Destination
1	7	6
2	2	8
2	6	5
4	2	5
4	0	1
6	8	6
7	7	8
7	2	3
8	0	7

Scanned with CamScanner

8 1 2

9 3 4

10 5 4

11 1 7

14 3 5

1. 7 — 6

2. 2 | 8

7 — 6

3. 2 | 8

7 — 6 — 5

4. 2 | 8

7 — 6 — 5

5. 1 2 | 0 8

7 — 6 — 5

6. 1 2 | 0 8

7 — 6 — 5

if form cycle discard it (8, 6)

7. 1 2 | 0 8

7 — 6 — 5

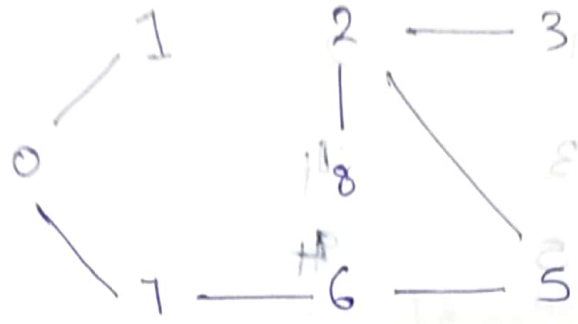
if form cycle discard

8. 1 2 — 3 | 0 8

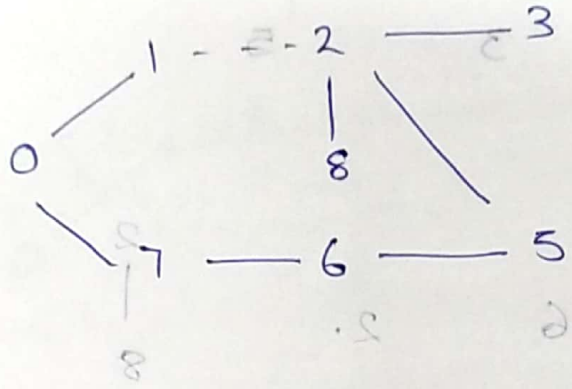
7 — 6 — 5

Scanned with CamScanner

9.

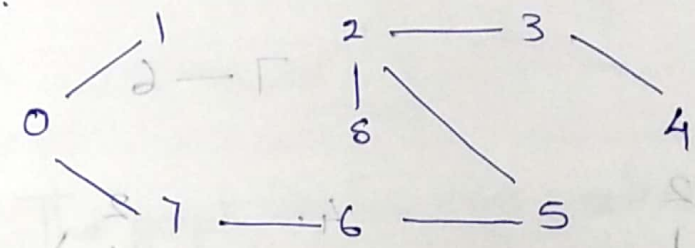


10.

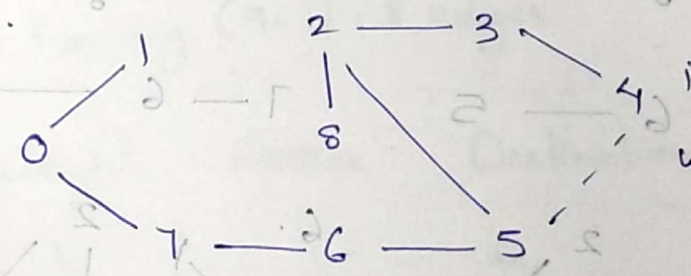


if found cycle it will discard (1,2)

11.

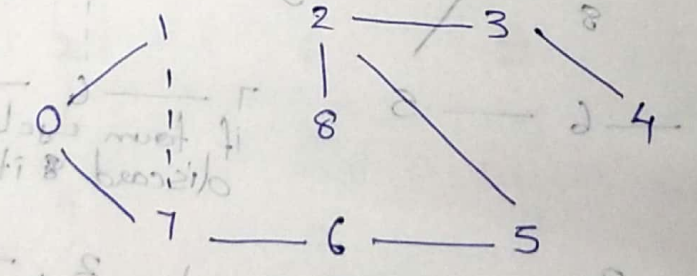


12.



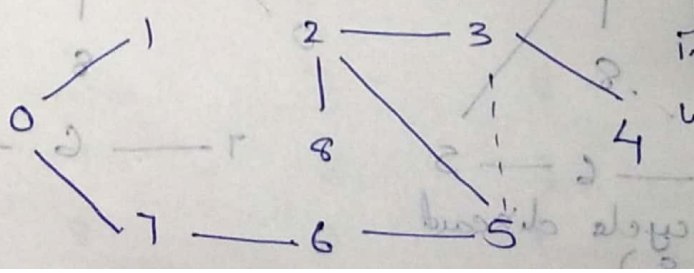
if found cycle it will discard (5,4)

13.



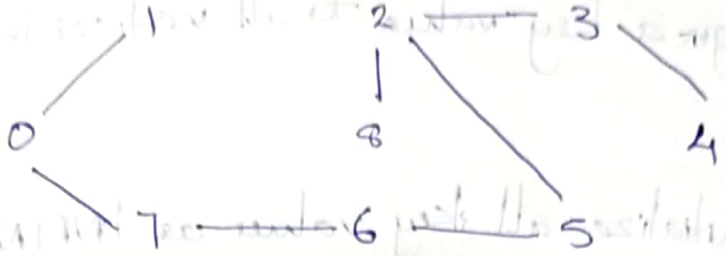
if found cycle it will discard (1,7)

14



if found cycle it will discard (3,5)

Scanned with CamScanner



Prim's Algorithm

It is also a greedy algorithm

It starts with an empty spanning tree

The idea is to maintain two sets of vertices

The first set contains the vertices already included in the MST, the other set contains the vertices not yet included.

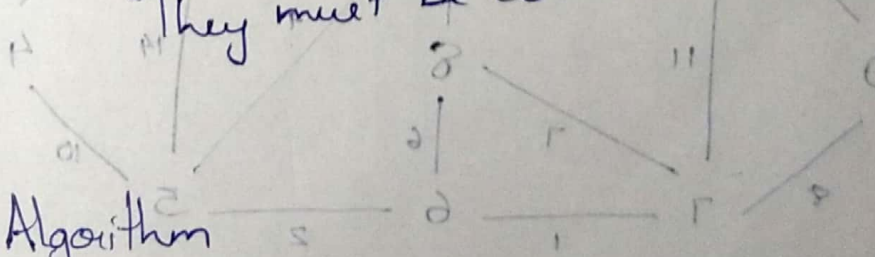
At every step, it considers all the edges that connected the two sets and picks the minimum weight edge from these edges

After picking the edge, it moves the other endpoint of the edge to the set containing MST.

The idea behind prim's algorithm is simple; a spanning tree means all vertices must be connected

So the two disjoint subsets of vab vertices must be connected to make a spanning tree

They must be connected with the minimum



Algorithm

1. Create a set mstSet that keeps track of vertices already included in MST.

Scanned with CamScanner

2. Assign a key value to all vertices in the input graph

Initialize all key values as INFINITE

Assign key value as 0 for the first vertex so that it is picked first.

3. While mstset doesn't include all vertices

a. Pick a vertex u which is not there in mstset and has minimum key value.

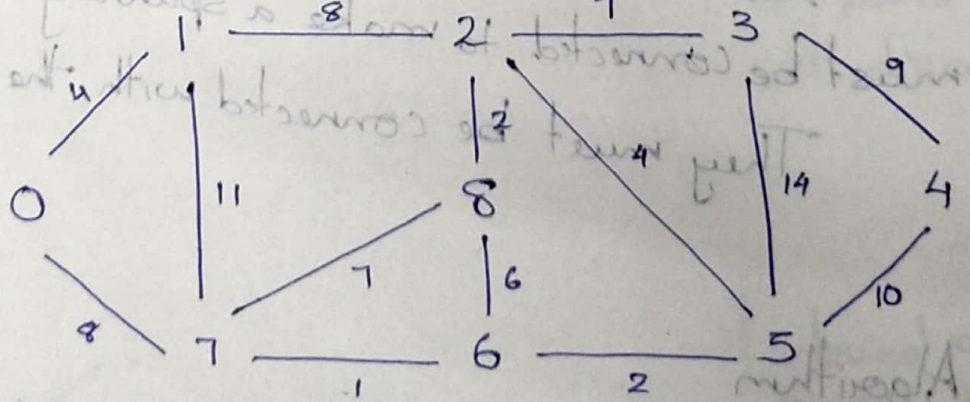
b. Include u to mstSet

c. Update key value of all adjacent vertices of u

To update the key values, iterate through all adjacent vertices.

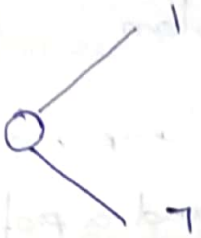
For every adjacent vertex v, if weight of edge u-v is less than the previous key value of v, update the key value as weight of u-v

Example

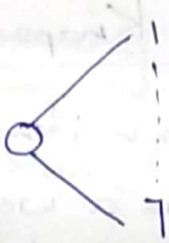


Scanned with CamScanner

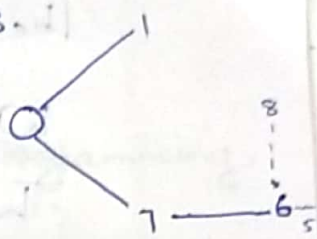
1.



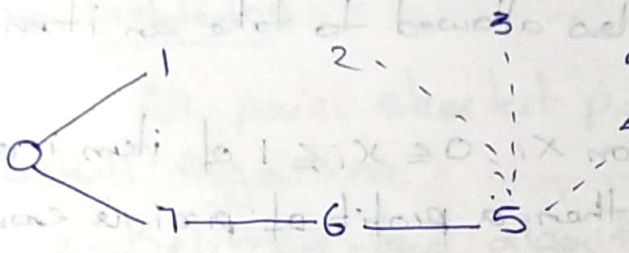
2.



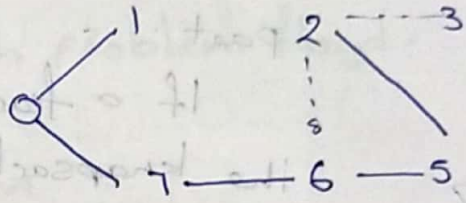
3.



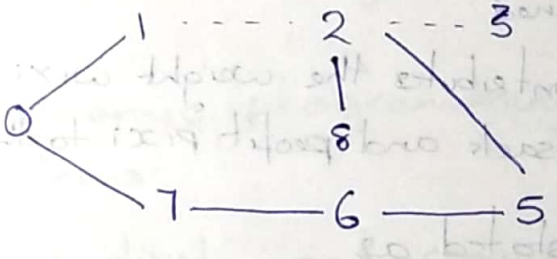
4.



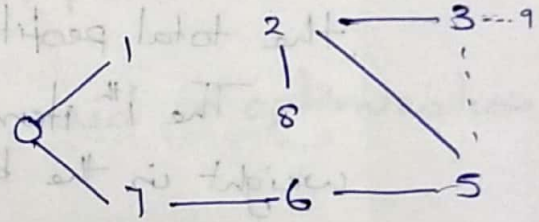
5.



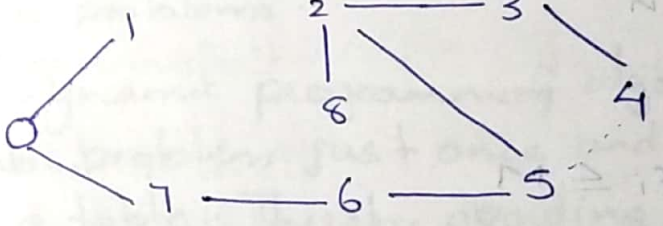
6.



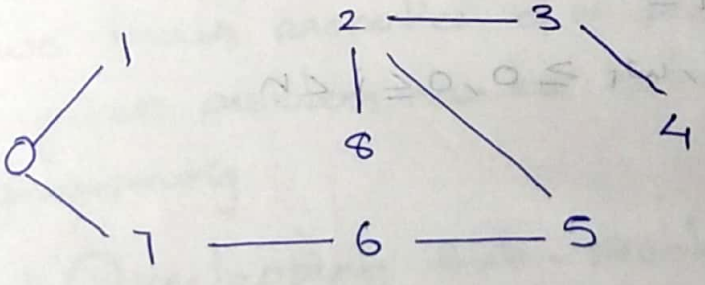
7.



8.



9.



Scanned with CamScanner

The Fractional Knapsack Problem

We are given n items and a knapsack. The item i has a weight w_i and a profit p_i. The knapsack has a capacity M.

We are also allowed to take an item in fraction part.

If a fraction x_i, $0 \le x_i \le 1$ of item i is placed into the knapsack then a profit of $p_i x_i$ is earned.

The objective is to fill the knapsack that maximizes the total profit earned.

The ith item contributes the weight $w_i x_i$ to the total weight in the knapsack and profit $p_i x_i$ to the total profit.

The problem is stated as

$$\max \sum_{0 \le i < n} p_i x_i$$

s.t.

$$\sum_{0 \le i < n} w_i x_i \le M$$

$$0 \le x_i \le 1$$

$$p_i \ge 0, w_i \ge 0, 0 \le i < n$$

Scanned with CamScanner

Dynamic Progeamming

Control abstraction - Dynamic progjamming
D and C VS DP

~~Proge~~ Problems

1. All-pairs shortest path problem floyd-warshall Algorithm.
2. Bellman-ford algorithm
3. Travelling sales person problem

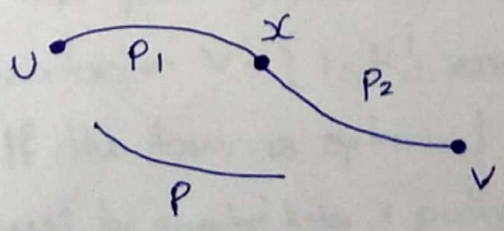
Dynamic progeamming is used in optimization problems.

Like divide-and-conquer method, Dynamic Programming solved problems by combining the solutions of sub problems.

Dynamic progeamming algorithm solves each sub-problem just once and then saves its answer in a table. Thereby avoiding the work of recomputing the answer every time.

Two main properties of a problem suggest that the given problem can be solved using Dynamic progeamming.

1. Overlapping sub-problems
2. Optimal sub-problems

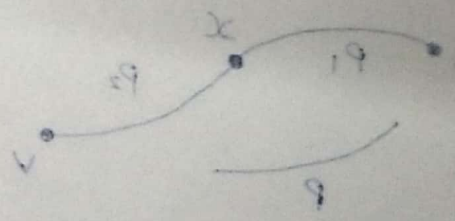


Scanned with CamScanner

Contd!

```
DP(n)
{
Optimal Solution OS[n] // data structure for optimal solution
for i ← 1 to n
OS[i] ← Compute (OS[1], OS[2] ... OS[j]), j<i
return OS[n]
}
```

Divide and Conquer	Dynamic
· recursive	· non recursive
· time consumption	· less time
· top-down approach	· Bottom-up approach
· Sub problem are independent of each other	· Sub problem are independent
· eg:- Merge sort and Binary search etc	· eg: Travelling Salesman



Scanned with CamScanner

Travelling Salesman Problem (TSP)

Given a set of cities and distance between every pair of cities, the problem is to find the shortest possible route that visits every city exactly once and returns to the starting point.

Problem:

- Let $G(V,E)$ be a directed graph with edge costs c_{ij}.
- The variable c_{ij} is defined such that $c_{ij} > 0$ for all i and j and $c_{ij} = \infty$ if $\langle i,j \rangle \notin E$.
- Let $|V| = n$ and assume $n > 1$.
- A tour of G is a directed simple cycle that includes every vertex in V.
- The cost of a tour is the sum of the cost of the edges on the tour.
- The travelling salesperson problem is to find a tour of minimum cost.

Solution:

- A tour - A simple path that starts and ends at vertex 1.
- Every tour consists of an edge $\langle 1, k \rangle$ for some $k \in V - \{1\}$ and a path from vertex k to 1.
- The path from vertex k to vertex 1 goes through each vertex in $V - \{1, k\}$ exactly once.
- If the tour is optimal, then the path from k to 1 must be shortest k-to-1 path going through all vertices in $V - \{1, k\}$. Hence the principle of optimality holds.

Scanned with CamScanner

· Let $g(i, S)$ be the length of
starting from i going through all vertices in S
and terminating at vertex 1.

· The function $g(1, V-\{1\})$ is the length of an optimal salesman tour.

· From the principle of optimality it follows that

$$g(1, V-\{1\}) = \min_{2 \le k \le n} \{c_{1k} + g(k, V-\{1, k\})\} \quad -(1)$$

Generalizing (1) we get (for $i \notin S$)

$$g(i, S) = \min_{j \in S} \{c_{ij} + g(j, S-\{j\})\} \quad -(2)$$

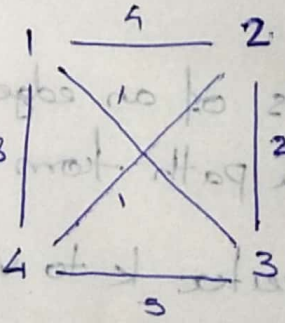
c_{ik} → it is also known as c_{ik}

· $g(1, V-\{1\})$ can be solved if we know $g(k, V-\{1, k\})$ for all choices of k.

· The g values can be obtained by using (2)

$$g(i, \phi) = c_{i1}, \quad 1 \le i \le n$$

Example



	1	2	3	4
1	0	4	1	3
2	4	0	2	1
3	1	2	0	5
4	3	1	5	0

$g(1, V-\{1\})$

$V = \{1, 2, 3, 4\}$

$\min(c_{1k} + g(k, V-\{1, k\}))$

Scanned with CamScanner

$g(1,\{2,3,4\})$ = minimum of

$= \{c_{12} + g(2,\{3,4\})$

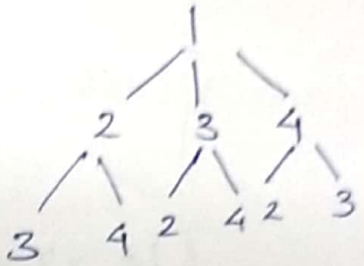
$= 4 + 6 = 10$

$= \{c_{13} + g(3,\{2,4\})$

$= 1 + 3 = 4$

$= \{c_{14} + g(4,\{2,3\})$

$= 3 + 3 = 6$



$g(2,\{3,4\})$ = minimum of

$= \{c_{23} + g(3,\{4\})$

$= 2 + 5 = 7$

$= \{c_{24} + g(4,\{3\})$

$= 1 + 5 = 6$

$g(3,\{2,4\})$ = minimum of

$= \{c_{32} + g(2,\{4\})$

$= 2 + 1 = 3$

$= \{c_{34} + g(4,\{2\})$

$= 5 + 1 = 6$

$g(4,\{2,3\})$ = minimum of

$= \{c_{42} + g(2,\{3\})$

$= 1 + 2 = 3$

$= \{c_{43} + g(3,\{2\})$

$= 5 + 2 = 7$

Scanned with CamScanner

$g(1,\{2,3,4\}) = \min(C_{12} + g(2,\{3,4\}))$

$= 4+6 = 10$

$= \min(C_{13} + g(3\{2,4\})$

$= 1+3 = 4$

$= \min(C_{14} + g(4,\{2,3\})$

$= 3+3 = 6$

$g(2\{3,4\}) = \min(C_{23} + g(3,\{4\}))$

$= 2+5 = 7$

$= \min(C_{24} + g(4,\{3\}))$

$= 1+5 = 6$

$g(3\{2,4\}) = \min(C_{32} + g(2,\{4\}))$

$= 2+1 = 3$

$= \min(C_{34} + g(4,\{2\}))$

$= 5+1 = 6$

$g(4,\{2,3\}) = \min(C_{42} + g(2,\{3\}))$

$= 1+2 = 3$

$= \min(C_{43} + g(3,\{2\}))$

$= 5+2 = 7$

$g(3\{4\}) = \min(C_{34} + g(4,\phi))$ $\quad g(i,\phi) = C_{i1}$

$= 5 + C_{41}$

$= 5+3 = 8$

$g(4,\{3\}) = \min(C_{43} + g(5,\phi))$

$= 5 + C_{31}$

$= 5+1 = 6$

Scanned with CamScanner

$g(2,\{4\}) = \min(C_{24} + g(4,\phi))$

$= 4 + C_{41}$

Shortest Path Problem

It is a problem is a problem of finding th shortest path (s) between vertices of a given grap

Shortest path between two vertices is a path that has the least cost as compared to o other existing paths.

i. Single - source shortest path problem
ii. All pairs shortest path problem

1. Single - Source Shortest Path Problem

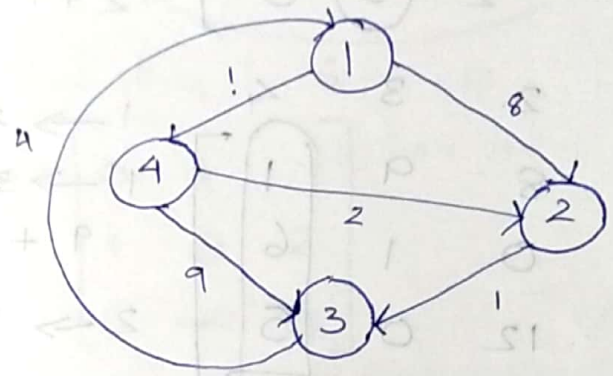
It is a shortest path problem where the shor path from a given source vertex to all other

Scanned with CamScanner

Floyd Warshall Algorithm

It is for solving the all pair shortest path problem

example



$D_0 =$

	1	2	3	4
1	0	8	∞	1
2	∞	0	1	∞
3	4	∞	0	∞
4	∞	2	9	0

$D_1 :$

	1	2	3	4
1	0	8	∞	1
2	∞	0	1	∞
3	4	12	0	5
4	∞	2	9	0

2 → 4

3 → 1 → 2

1 + 4 + 1 = 6

$D_2 :$

	1	2	3	4
1	0	8	∞	1
2	∞	0	1	∞
3	4	12	0	5
4	∞	2	9	0

3 → 2

3 → 1 → 2

4 + 8 = 12

3 → 4

3 → 1 → 4

4 + 1 = 5

Scanned with CamScanner

D_3 :

	1	2	3	4
1	0	8	9	1
2	∞	0	1	∞
3	4	12	0	5
4	∞	2	3	0

$1 \to 2 \to 3$

$8 + 1 = 9$

$4 \to 3$

$4 \to 2 \to 3$

$2 + 1 = 3$

D_4 :

	1	2	3	4
1	0	8	9	1
2	5	0	1	6
3	4	12	0	5
4	7	2	3	0

$1 \to 2$

$1 \to 3 \to 2$

$9 +$

$2 \to 1$

$2 \to 3 \to 1$

$1 + 4 = 5$

D_5 :

	1	2	3	4
1	0	8	4	1
2	5	0	1	6
3	4	12	0	5
4	7	2	3	0

$2 \to 4$

$2 \to 3 \to 4$

$1 + 5 = 6$

$4 \to 1$

$4 \to 3 \to 1$

$3 + 4 = 7$

$1 \to 2$

$1 \to 4 \to 2$

$1 + 2 = 3$

$3 \to 1$

$3 \to 4 \to 1$

$5 +$

$1 \to 3$

$1 \to 4 \to 3$

$1 + 3 = 4$

$3 \to 2$

$3 \to 4 \to 2$

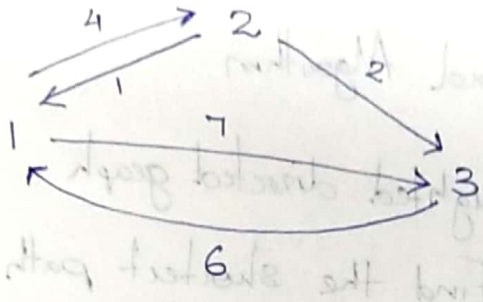
$5 + 2 = 7$

$2 \to 1$

$2 \to 4 \to 1$

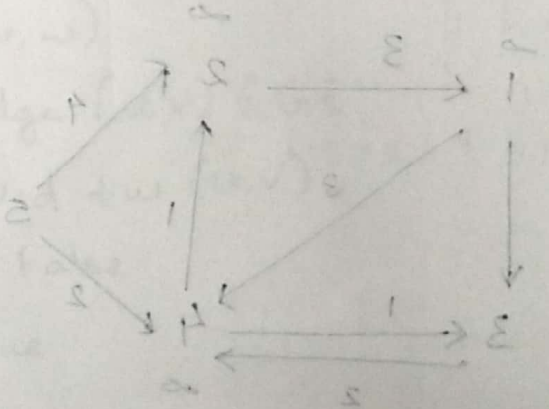
$6 + 2 = 8$

Scanned with CamScanner



$$D_0 = \begin{bmatrix} 0 & 4 & 7 \\ 1 & 0 & 2 \\ 6 & \infty & 0 \end{bmatrix}$$

$$D_1 = \begin{matrix} D & 4 & 7 \\ 1 & 0 & 2 \\ 6 & 10 & 0 \end{matrix}$$



Single Source Shortest Path

Bellman Ford Algorithm

Input : Weighted directed graph

Problem : Find the shortest path from a source vertex to all other vertices

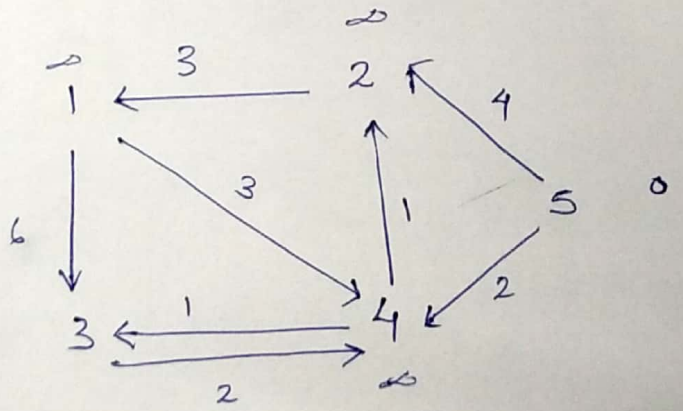
$G(V, E)$

$S \rightarrow v \in V$

Path $\rightarrow P$

$$\delta(s, v) = \min \begin{cases} \min(w(p)) \\ \infty \quad \text{otherwise} \end{cases}$$

$$w(p) = \sum_{i=1}^{n} (c(v_{i-1}, v_i))$$



Scanned with CamScanner

Algorithm

Bellman_Ford (G, w, s)

1. Initialize_Single Source (G, s)
2. for i = 1 to |G.V| - 1
3. for each edge (u, v) ∈ G.E
4. Relax (u, v, w)
5. for each edge (u, v) ∈ G.E
6. if v.d > u.d + w (u, v)
   return False
7. Return True

v.d → distance to vertex v from same vertex

v.π → Predecessor of vertex v along the path from s

G (V, E)

Initialize_Single_Source (G, s)

for each vertex v ∈ G.V

v.d = ∞

v.π = NIL u → v

s.d = 0

Relax (u, v, w)

if v.d → u.d + w(u, v)

Scanned with CamScanner

$v.d = u.d + w(u, v)$

~~v.d~~ $v.\pi = u$

Procedure Backtrack (n)

{

k = 1

while k > 0 do

if there remained an untried x(k) such that x(k) ∈ T(x(1) .. x(k-1)) and B_k x(1) ... x(k)) : true then if (x(1) ... x(k)) is a path to an answer node then print (x(1) ... x(k))

end if

k = k + 1 // Consider next set //

else k = k - 1 // backtrack to previous set

end if

repeat

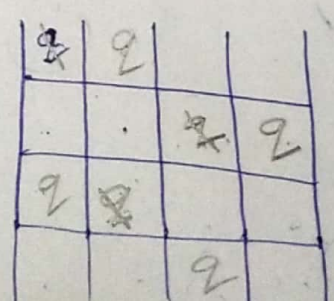
end Backtrack

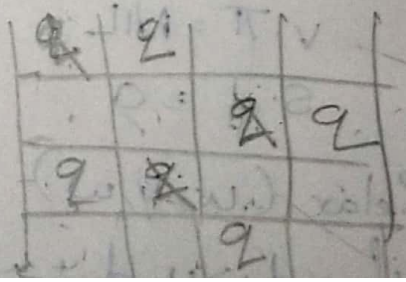
}

N-Queens Problem

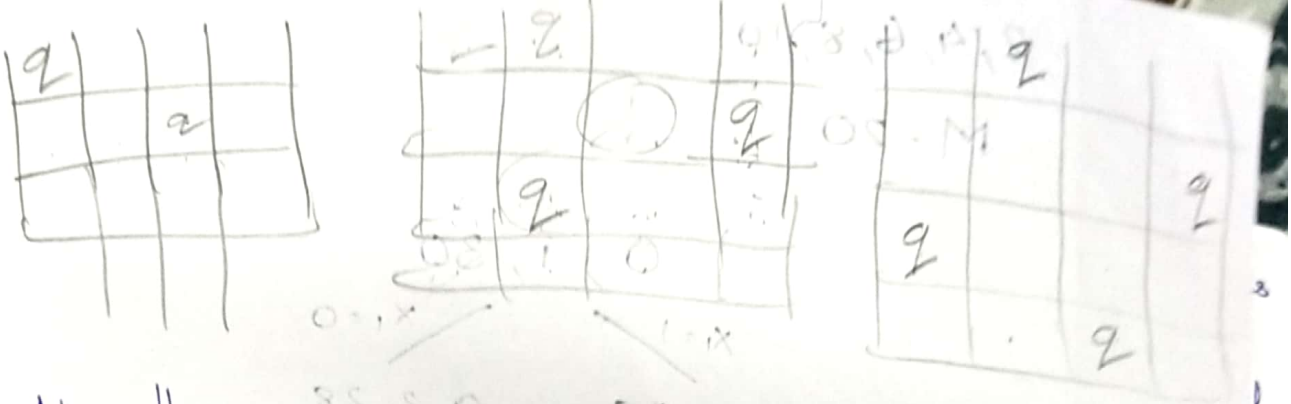
Problem

N-Queens problem is to place n-queens in such a manner on an n×n chessboard that no queens





Scanned with CamScanner



Algorithm

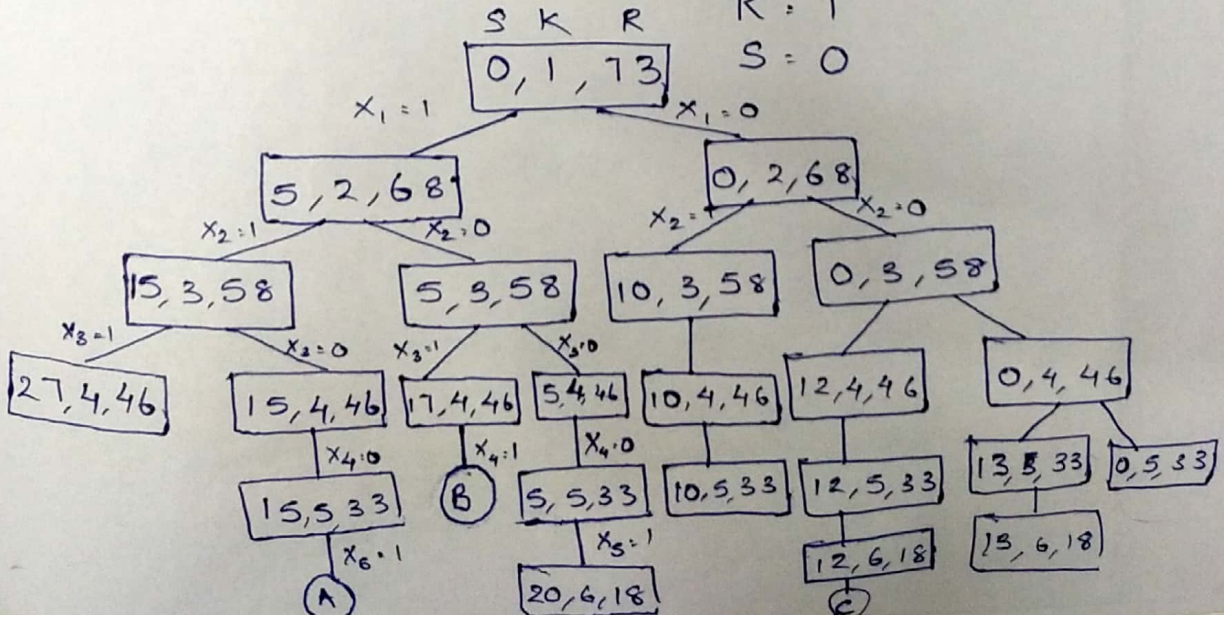
```
N-Queens (k, n)
{
 for i ← 1 to n
  do if Place (k, i) then
 {
  x[k] ← i;
  if (k==n) then
  write (x[1...n]);
  else
  N-Queens (k+1, n);
 }
}
```

Sum-of-Subset Problem

5, 10, 12, 13, 15, 18 R = 75

K = 1

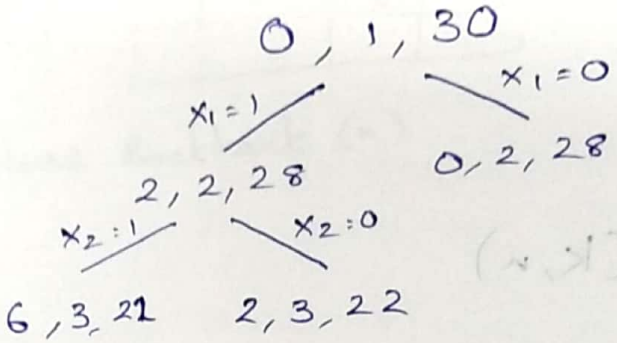
S = 0

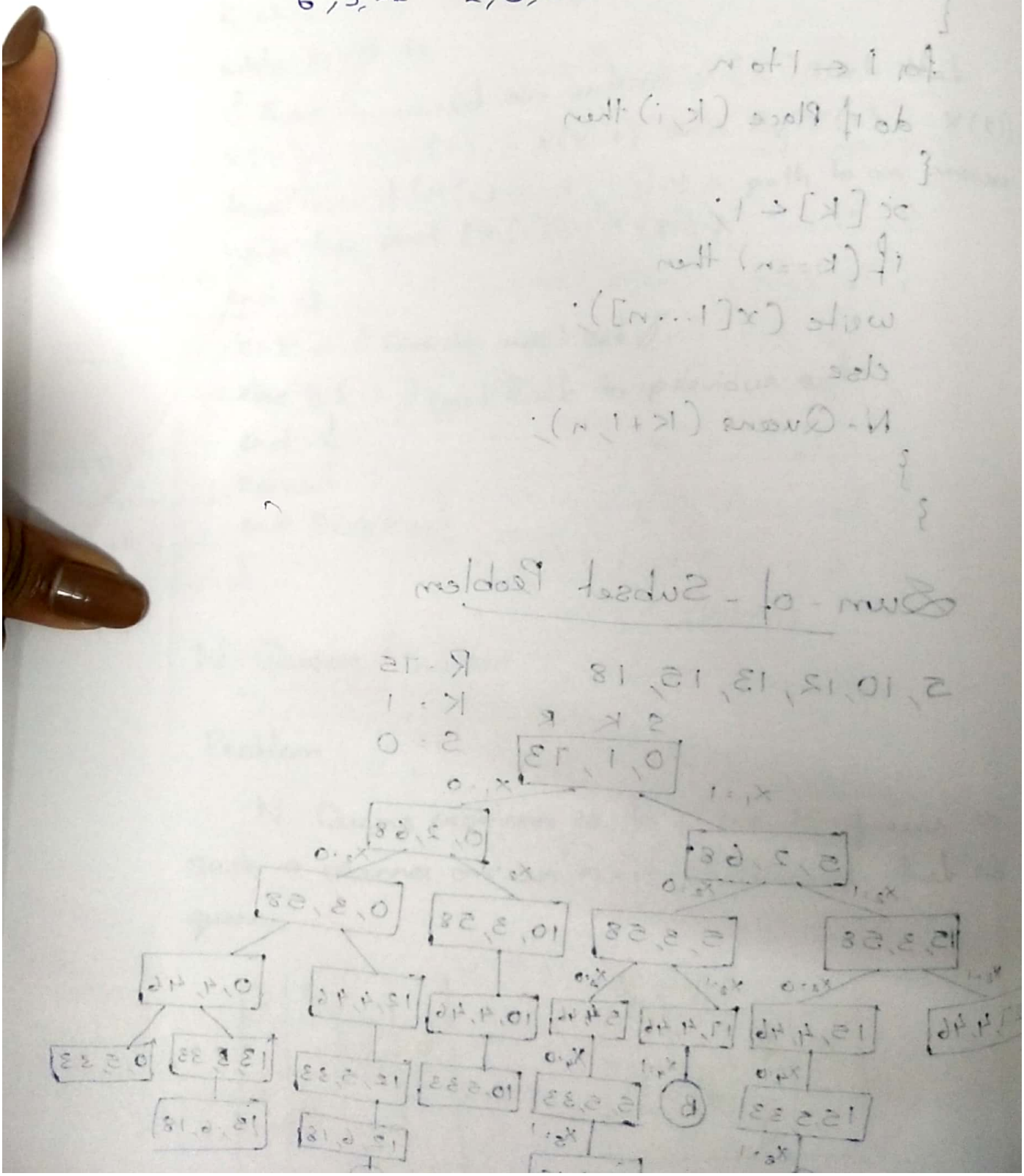


Scanned with CamScanner

2, 4, 6, 8, 10

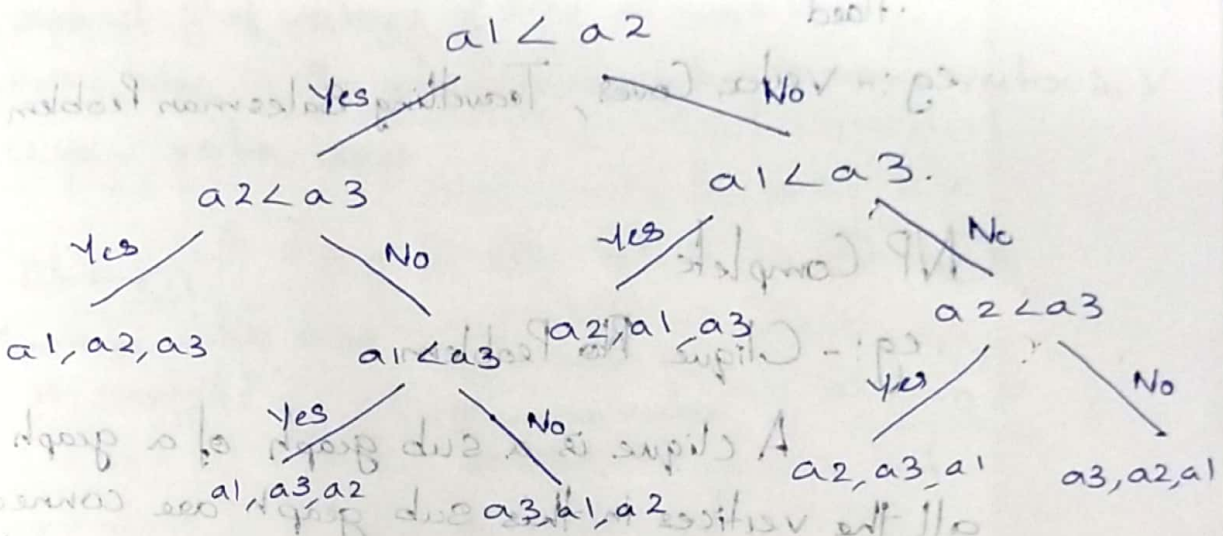
M = 20





Module 3

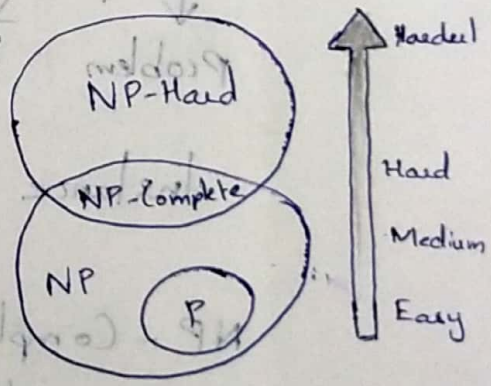
Lower Bound



Module 4

Complexity Classes

1. Optimization Problem
2. Decision Problem



Classes of Problems:

1. P - Class - quick to solve
2. NP - Class - quick
3. NP - Hard
4. NP - Complete

NP - Hard and NP - Complete Problem

We have 2 problem A and B and we know problem B is a P class problem

NP Hard

A problem is NP-hard all problems in NP are polynomial time

- Hard

eg:- Vertex Cover, Travelling Salesman Problem

NP Complete

eg:- Clique The Problem -

A clique is a sub graph of a graph such that all the vertices in this sub graph are connected with each other that is the sub graph is a complete graph

NP Hard

$A \leq_P B$

↓ Problem ↓ Problem

SAT → Clique

↓

Satisfiability Problem

Instance of A → input instance of B → Algorithm B (Yes / No)

NP - Complete

NP - Verification

S = { }

Subgraph → complete } NP

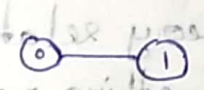
K - clique size

Clique is a maximum problem

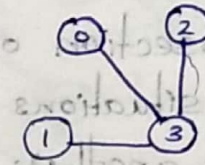
Scanned with CamScanner

Vertex Cover Problem

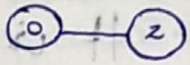
Graph G(V,E)
positive integer k
subset V of vertices of size at most k
every edge in the graph is connected to some vertex in V
u or v vertex cover



Minimum vertex cover is empty ( )



Minimum vertex cover is (3)



Module 4

Network Flow

It is a directed graph that is used to model material flow. It involves a source (s) and a sink (t) and several other nodes connected with each other.

It is a subsection of graph theory related specifically to situations where something moves from one location to another.

An easily visualized example of network flow is piping system through which a quantity of water must pass.

- A Network is a directed graph G
- Edges represent pipes that

Two different vertices

- Source
- Sink

$G = (V, E)$ directed graph

edge $(u, v) \in E$

Capacity $c(u, v) \geq 0$

$(u, v) \notin E$ we assume that $c(u, v) = 0$

two distinguishing points, source s and sink t

vertex $v \in V$

path from s to t containing v

real-valued function $f : V \times V \rightarrow R$

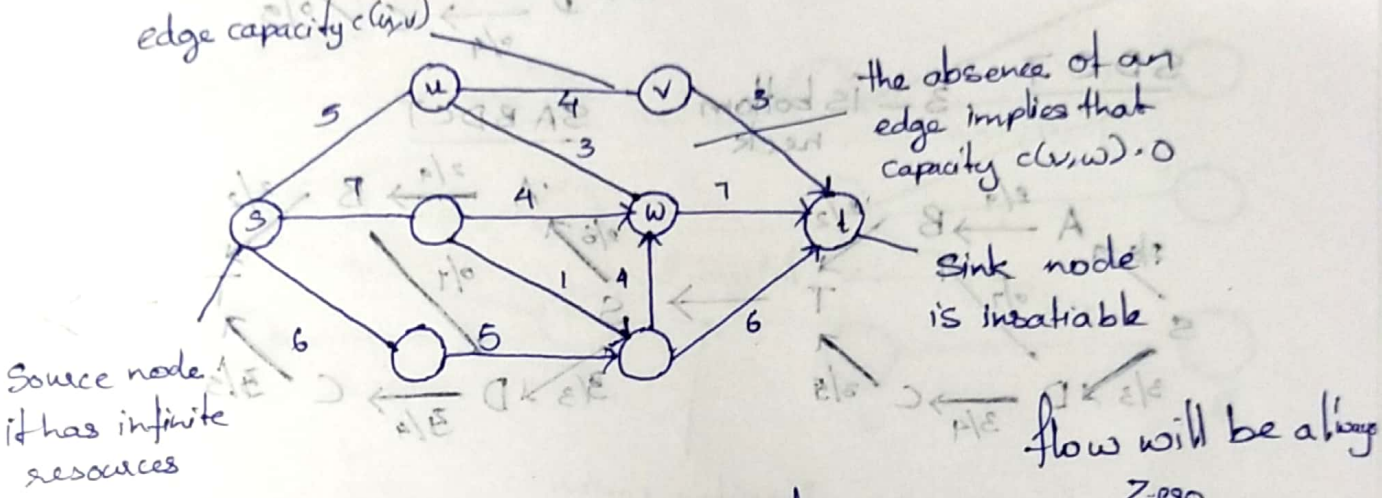
Capacity Constraint: for all $u, v \in V$, we need $f(u,v) \leq c(u,v)$

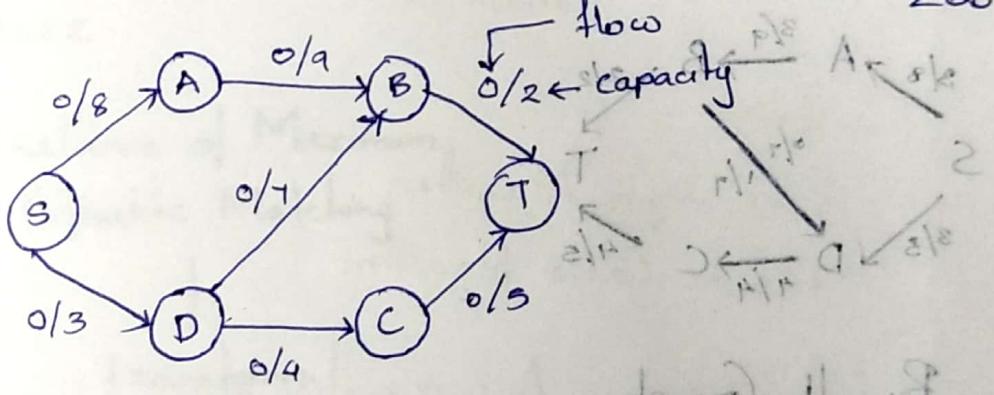
Scanned with CamScanner

Skew Symmetry : For all $u \in V$, we need $f(u,v) f(u,v)$

Flow Conservation : for all $u \in V - \{s, t\}$

$$\sum_{v \in V} f(u,v) = \sum_{u \in V} f(u,v) = 0$$





Ford - Fulkerson Algorithm

It is a greedy approach for calculating the maximum possible flow in a network or a graph.

Max - Flow - Maximum flow in the given graph

Terminologies Used

1. Augmenting Path
2. Residual Graph
3. Residual Capacity

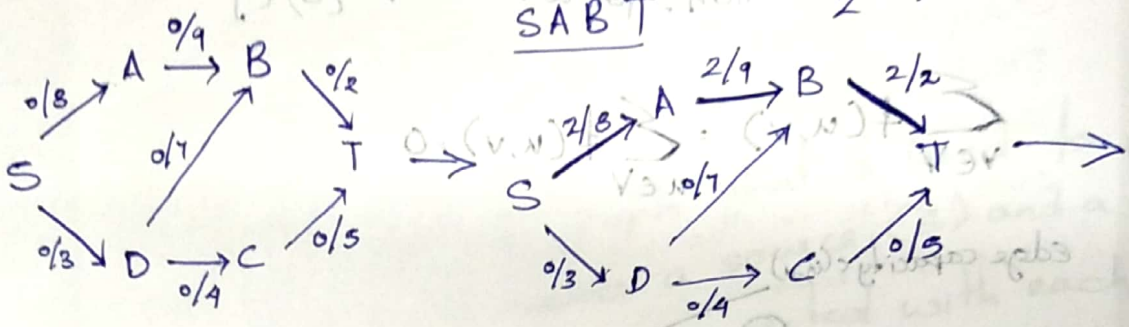
Algorithm

1. Initialize the flow in all edges to 0
2. While there is an augmenting path between the source and sink, add this path to the flow.

Scanned with CamScanner

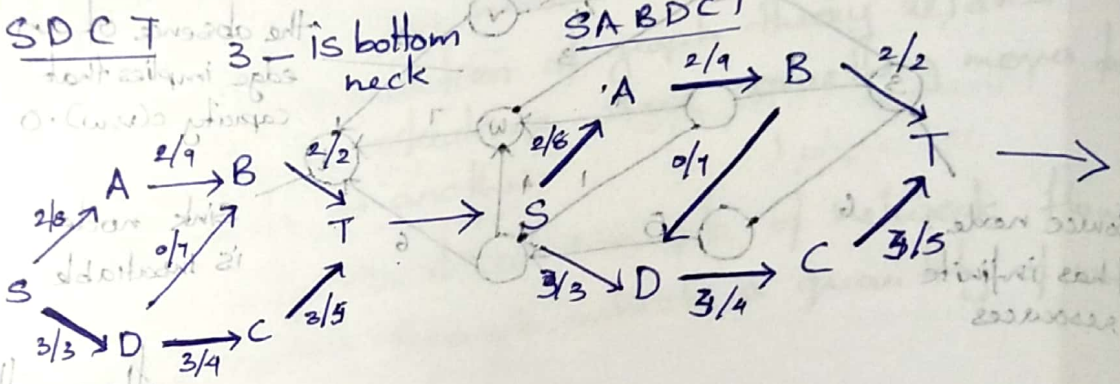
3. Update the residual graph

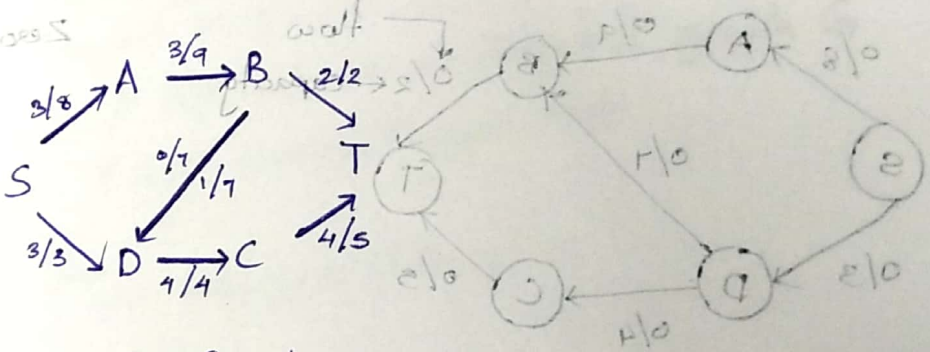
SABT — 2 is bottom neck



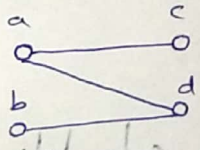
SDCT — 3 is bottom neck

SABDCT





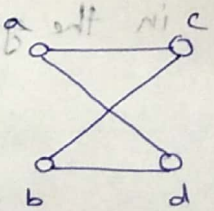
Bipartite Graph



$V = \{a, b, c, d\}$

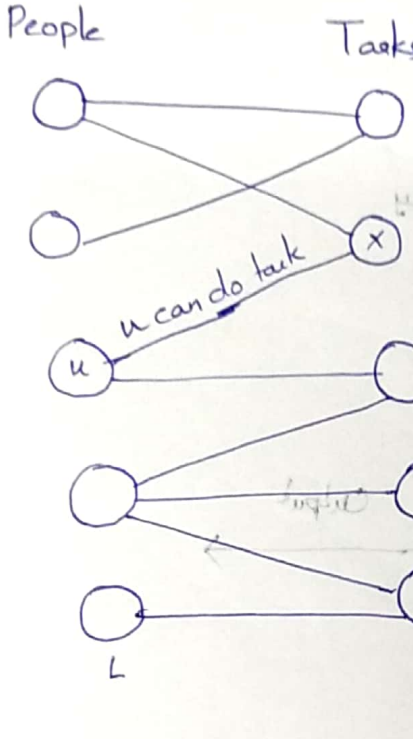
V_1	V_2
a	c
b	d

Complete Bipartite Graph

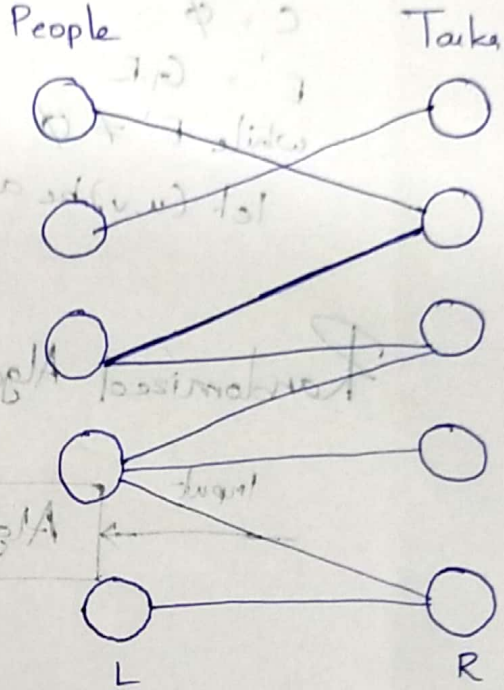


Scanned with CamScanner

Bipartite Graphs



Bipartite Matching



Reduce

Instance of Maximum Bipartite Matching

|

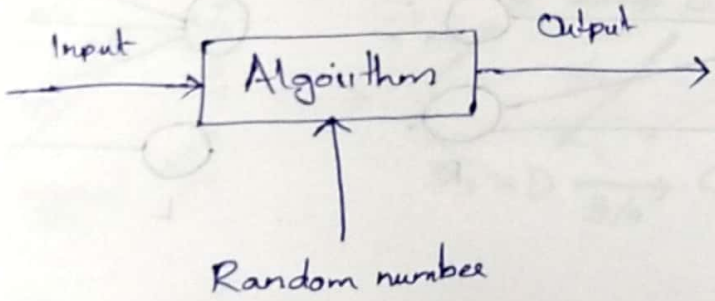
transform aka reduce

|

Approx - Vertex Cover

C = ∅
E' = G.E
while E' ≠ ∅
  let (u,v) be an arbitrary

Randomized Algorithm



2 types
- Las Vegas Algorithm
- Monte Carlo Algorithm

Vertex Cover Approximation using Linear Programming

Undirected graph G = (V,E)
subset S ⊆ V Call the edges have atleast one vertex S
weight given on every vertex $(w_v, \forall, v \in V)$
Minimize the weight S, $w(S) = \sum_{v \in S}$

Integer Program

$\min \sum_{v \in V} w_v y_v$
s.t. $y_u + y_v \geq 1 \quad \forall \{u,v\} \in E$
$y_v \in \{0,1\} \quad \forall v \in V$

Scanned with CamScanner